SQLite to Structured JSON

Parse SQLite databases by exploring schemas first, then extracting data into structured JSON.

Step 1: Explore the Schema

Always start by understanding what tables exist and their structure.

List All Tables
```sql
SELECT name FROM sqlite_master WHERE type='table';
```

Inspect Table Schema
```sql
-- Get column names and types
PRAGMA table_info(TableName);

-- See CREATE statement
SELECT sql FROM sqlite_master WHERE name='TableName';
```

Find Primary/Unique Keys
```sql
-- Primary key info
PRAGMA table_info(TableName);  -- 'pk' column shows primary key order

-- All indexes (includes unique constraints)
PRAGMA index_list(TableName);

-- Columns in an index
PRAGMA index_info(index_name);
```

Step 2: Understand Relationships

Identify Foreign Keys
```sql
PRAGMA foreign_key_list(TableName);
```

```

### Common Patterns

**ID-based joins:** Tables often share an ID column
```sql
-- Main table has ID as primary key
-- Related tables reference it
SELECT m.*, r.ExtraData
FROM MainTable m
LEFT JOIN RelatedTable r ON m.ID = r.ID;
```

Coordinate-based keys: Spatial data often uses computed coordinates
```python
# If ID represents a linear index into a grid:
x = id % width
y = id // width
```

Step 3: Extract and Transform

Basic Pattern
```python
import sqlite3
import json

def parse_sqlite_to_json(db_path):
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row  # Access columns by name
    cursor = conn.cursor()

    # 1. Explore schema
    cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
    tables = [row[0] for row in cursor.fetchall()]

    # 2. Get dimensions/metadata from config table
    cursor.execute("SELECT * FROM MetadataTable LIMIT 1")
    metadata = dict(cursor.fetchone())

    # 3. Build indexed data structure
    data = {}
    cursor.execute("SELECT * FROM MainTable")
    for row in cursor.fetchall():
        key = row["ID"]  # or compute: (row["X"], row["Y"])
        data[key] = dict(row)

    # 4. Join related data
    cursor.execute("SELECT * FROM RelatedTable")
    for row in cursor.fetchall():
        key = row["ID"]
        if key in data:
            data[key]["extra_field"] = row["Value"]

    conn.close()
    return {"metadata": metadata, "items": list(data.values())}
```
```

Handle Missing Tables Gracefully
```python
def safe_query(cursor, query):
    try:
        cursor.execute(query)
        return cursor.fetchall()
    except sqlite3.OperationalError:
        return []  # Table doesn't exist
```

Step 4: Output as Structured JSON

Map/Dictionary Output
Use when items have natural unique keys:
```json
{
  "metadata": {"width": 44, "height": 26},
  "tiles": {
    "0,0": {"terrain": "GRASS", "feature": null},
    "1,0": {"terrain": "PLAINS", "feature": "FOREST"},
    "2,0": {"terrain": "COAST", "resource": "FISH"}
  }
}
```

Array Output
Use when order matters or keys are simple integers:
```json
{
  "metadata": {"width": 44, "height": 26},
  "tiles": [
    {"x": 0, "y": 0, "terrain": "GRASS"},
    {"x": 1, "y": 0, "terrain": "PLAINS", "feature": "FOREST"},
    {"x": 2, "y": 0, "terrain": "COAST", "resource": "FISH"}
  ]
}
```

Common Schema Patterns

Grid/Map Data
- Main table: positions with base properties
- Feature tables: join on position ID for overlays
- Compute (x, y) from linear ID: `x = id % width, y = id // width`

Hierarchical Data
- Parent table with primary key
- Child tables with foreign key reference
- Use LEFT JOIN to preserve all parents

Enum/Lookup Tables
- Type tables map codes to descriptions
- Join to get human-readable values

Debugging Tips

```sql
-- Sample data from any table
```

```
SELECT * FROM TableName LIMIT 5;

-- Count rows
SELECT COUNT(*) FROM TableName;

-- Find distinct values in a column
SELECT DISTINCT ColumnName FROM TableName;

-- Check for nulls
SELECT COUNT(*) FROM TableName WHERE ColumnName IS NULL;
```